CTO Master Technical Report

GPS Installation Automation System

1) Executive Technical Summary

This project is a role-based workflow platform for GPS installation operations.
It orchestrates request lifecycle management across FO, RH, Payment, and Vendor using: - React + Vite frontend - Firebase Authentication - Cloud Firestore (real-time state + audit history) - Firebase Cloud Functions (email/notification orchestration)

The system supports both **single-request** and **bulk-request** workflow models with strict transition controls and role-based authorization.

---

2) Technology Stack

Frontend

- React `19.x`
- React Router `7.x`
- TypeScript/JS mixed codebase
- Vite `7.x`
- XLSX export support (`xlsx`)

Backend / Platform

- Firebase Auth
- Firestore
- Firebase Functions v2 (HTTP)
- Node mail transport via `nodemailer`
- CORS-enabled HTTPS endpoints for notification services

---

3) System Architecture

Layered design

1. **Presentation Layer**: role dashboards + reusable components.
2. **Context Layer**: auth/session/role orchestration.
3. **Service Layer**:
   - `requestService` (Firestore operations + workflow updates)
   - `workflowService` (state transition engine)
   - `functionsService` (HTTP calls to cloud functions)
4. **Backend Layer**:
   - Cloud Functions for OTP + vendor/FO notification flows
   - Firestore as source of truth

Dashboard routing model

- `/fo-dashboard` (FO)
- `/rh-dashboard` (RH)
- `/payment-dashboard` (PAYMENT)
- `/vendor-dashboard` (VENDOR)

Protected route logic enforces role checks before rendering each dashboard.

---

4) Workflow Engine (Core Business Logic)

Workflow logic is centralized in `workflowService.updateRequestState()` and invoked by `requestService` write methods.

4.1 Workflow execution model

- Every state transition is generated by `updateRequestState(request, action, user, optionalData)`.
- `requestService` performs document reads/writes and delegates status logic to workflow service.
- Each action appends a `history` entry with actor, role, from/to status, timestamp, and notes.
- Transition failures are fail-fast (explicit error thrown; no partial status mutation).

4.2 Single-request lifecycle (non-bulk)

Step A: Request creation

- Action: `CREATE`
- Preconditions: request does not already exist.
- Resulting state:
  - `status = PARALLEL_REVIEW`
  - `rhApproval = false`
  - `paymentApproval = false`
  - `rhStatus = PENDING`
  - `paymentStatus = PENDING`
  - `vendorStatus = PENDING`

Step B: RH compliance action

- Approve action: `RH_APPROVE` or `RH_EDIT_APPROVE`
- Allowed statuses: `PARALLEL_REVIEW`, `VENDOR_COORDINATION`, `COMPLETED` (compliance can be recorded without rolling back business stage).
- Resulting field updates:
  - `rhApproval = true`
  - `rhActionTaken = true`
  - `rhStatus = APPROVED`
  - `rhApprovedAt = serverTimestamp()`
- Reject action: `RH_REJECT`
- Allowed status: `PARALLEL_REVIEW`
- Mandatory input: `rejectionReason`
- Resulting state: `status = HALTED`, `rhStatus = REJECTED`

Step C: Payment financial action

- Approve action: `PAYMENT_APPROVE` or `PAYMENT_EDIT_APPROVE`
- Allowed status: `PARALLEL_REVIEW`
- Resulting state:
  - `paymentApproval = true`
  - `paymentActionTaken = true`
  - `paymentStatus = APPROVED`
  - `paymentApprovedAt = serverTimestamp()`
  - `status = VENDOR_COORDINATION`
- Reject action: `PAYMENT_REJECT`
- Allowed status: `PARALLEL_REVIEW`
- Mandatory input: `rejectionReason`
- Resulting state: `status = HALTED`, `paymentStatus = REJECTED`

Step D: Vendor coordination completion

- Action: `VENDOR_NOTIFY`
- Allowed status: `VENDOR_COORDINATION`
- Mandatory input: `vendorName`
- Resulting state:
  - `status = COMPLETED`
  - `vendorName = <selected vendor>`
  - `notificationTimestamp = serverTimestamp()`
  - `vendorApprovedAt = serverTimestamp()`
  - `vendorApprovedBy = <actor uid>`

4.3 Bulk-request lifecycle (`isBulkRequest = true`)

Step A: Bulk creation

- Action: `CREATE` with `isBulkRequest = true`
- Resulting state:
  - `status = FO_CREATED`
  - `rhStatus = PENDING`
  - `paymentStatus = PENDING`
  - `vendorStatus = PENDING`
  - bulk tracking fields available: `vehicleCount`, per-vehicle records, payment/vendor per-vehicle flags.

Step B: RH bulk compliance

- Approve action: `RH_BULK_APPROVE`
- Allowed statuses: `FO_CREATED`, `PAYMENT_PENDING`, `PAYMENT_APPROVED`, `SERVICE_INITIATED`, `COMPLETED`
- Resulting updates:
  - `rhStatus = APPROVED`
  - `rhApprovedAt = serverTimestamp()`
  - `bothApproved = (paymentStatus === APPROVED)`
  - business `status` remains current stage (no backward/forward jump)
- Reject action: `RH_BULK_REJECT`
- Allowed status: `FO_CREATED`
- Mandatory input: `rejectionReason`
- Resulting state: `status = HALTED`, `rhStatus = REJECTED`

Step C: Payment bulk financial approval

- Approve action: `PAYMENT_BULK_APPROVE`
- Allowed statuses: `FO_CREATED`, `PAYMENT_PENDING`
- Resulting updates:
  - `paymentStatus = APPROVED`
  - `paymentApprovedAt = serverTimestamp()`
  - `bothApproved = (rhStatus === APPROVED)`
  - `status = PAYMENT_APPROVED`
- Reject action: `PAYMENT_BULK_REJECT`
- Allowed statuses: `FO_CREATED`, `PAYMENT_PENDING`
- Mandatory input: `rejectionReason`
- Resulting state: `status = HALTED`, `paymentStatus = REJECTED`

Step D: Vendor bulk notification

- Action: `VENDOR_BULK_NOTIFY`
- Allowed status: `PAYMENT_APPROVED`
- Mandatory input: `vendorName`
- Resulting state:
  - `vendorStatus = NOTIFIED`
  - `vendorName = <selected vendor>`
  - `vendorApprovedAt = serverTimestamp()`

    - `vendorApprovedBy = <actor uid>`
    - `status = SERVICE_INITIATED`

4.4 Operational guardrails

- Illegal status/action combinations are blocked via `ensureStatus` validation.
- Required fields are enforced via `ensureRequiredField` validation.
- Role identity is enforced before transition (`ensureRole`).
- All mutations write `updatedAt` and append immutable `history` records.
- Request write path sanitizes phone fields to strict 10-digit storage format.

---

5) Role and Authorization Model

Authentication

- Email/password auth via Firebase Auth.
- Registration path uses OTP email call before account creation in UI flow.

Authorization

- Role is resolved from Firestore `users/{uid}` document.
- Frontend protected routes deny mismatched roles.
- Function-level role checks are applied for sensitive endpoints (e.g., vendor coordinator-only operations using bearer token + role validation).

Supported roles

- FO
- RH
- PAYMENT
- VENDOR

---

6) Firestore Data Model

Core collections

1. `users`
    - `email`, `role`, `createdAt`, optional login metadata.
2. `requests`
    - Identity and metadata: `id`, `createdBy`, timestamps.
    - Request data: city/client/service details, vehicle arrays, LTPOC/LPO fields.
    - Workflow state fields:
        - `status`
        - single-path flags (`rhApproval`, `paymentApproval`, etc.)
        - bulk-path flags (`rhStatus`, `paymentStatus`, `vendorStatus`, etc.)
    - Notification fields:
        - `vendorNotified`, `foNotified`, `notificationTimestamp`
    - Audit fields:
        - `history[]` with user, role, action, status transition, timestamp, notes.

Data quality controls

- Phone values are sanitized/validated to strict 10-digit format in request write paths.
- Workflow updates are transactional-style and centralized to prevent divergence.

---

7) API / Function Layer

Frontend service `functionsService` calls HTTP endpoints using `FUNCTIONS_BASE_URL` (env-driven with local dev fallback).

Key endpoints

- `sendOTP`
- `sendVendorNotification`
- `sendVendorBulkNotification`
- `sendFoBulkNotification` (supports auth token header)

Function backend behaviors

- SMTP configuration via environment variables.
- Real recipient routing with optional test override.
- Payload validation + duplicate prevention checks.
- Firestore-backed coordination for request-level notification state.

---

8) Dashboard Capability Matrix

FO

- Create/manage own requests.
- Search, inspect, cancel, and remove vehicles from bulk requests at allowed stages.

RH

- Review all requests.
- Approve/reject single or bulk requests.
- Execute bulk approvals with eligibility checks.

Payment

- Financial review of request/vehicle rows.
- Approve/reject with mandatory reasoning on rejection.
- Filter-heavy operational queue management.

Vendor Coordinator

- Determine vendor-eligible records.
- Trigger vendor notifications.
- Export operational sheets.
- Trigger FO notifications after vendor processing.

---

9) Operational Flow and Real-Time Behavior

- Dashboards subscribe to Firestore snapshots (`onSnapshot`) for real-time updates.
- UI state reflects backend changes without manual refresh.
- Every role's action contributes to a shared workflow record and audit history.

This design supports high visibility while minimizing coordination lag between teams.

---

10) Environment and Configuration

Frontend environment requirements

- Firebase config variables (`VITE_FIREBASE_*`)
- Optional function base URL override (`VITE_FUNCTIONS_BASE_URL`)
- Optional emulator toggle (`VITE_USE_EMULATORS=true`)

Build/runtime scripts

- `npm run dev`
- `npm run build`
- `npm run preview`

Deployment artifacts present

- `firebase.json`
- Firestore indexes + rules files
- `functions/` package for Cloud Functions deployment target

---

11) Key Risks and Controls

Risks

- Role mismatch in `users` doc can block routing.
- SMTP misconfiguration can break OTP/notification operations.
- Workflow misuse can be attempted from stale UI state.

Controls implemented

- Strict route guards + role checks.
- Backend payload validation and permission checks.
- Centralized status transition validation.
- Audit logging for all workflow actions.